

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №9**  
**по дисциплине «Вычислительная математика»**  
**Тема: Составные формулы прямоугольников, трапеций, Симпсона.**

Студент гр. 4342

Озернов Е.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

## **Цель работы**

Цель лабораторной работы — изучить численное вычисление определённого интеграла с помощью квадратурных формул прямоугольников, трапеций и Симпсона. В ходе работы требуется реализовать вычисление заданной подынтегральной функции и программные функции для нахождения интеграла каждым из указанных методов. Также необходимо применить правило Рунге для оценки погрешности и подобрать наименьшее значение  $n$ , то есть наибольшее значение шага  $h$ , при котором приближённое значение интеграла будет найдено с заданной точностью  $\varepsilon$ . Дополнительно требуется сравнить работу формул прямоугольников, трапеций и Симпсона по полученным значениям интеграла, числу разбиений  $n$  и точности результата.

## Задание

Повышения точности численного интегрирования добиваются путем применения составных формул. Для этого при нахождении определенного интеграла отрезок  $[a, b]$  разбивают на четное  $n = 2m$  число отрезков длины  $h = (b - a) / n$  и на каждом из отрезков длины  $h$  применяют соответствующую формулу. Таким образом получают составные формулы прямоугольников, трапеций и Симпсона.

На сетке  $x_i = a + ih$ ,  $y = f(x_i)$ ,  $i = 0, 1, 2, \dots, 2m$ , составные формулы имеют следующий вид:

Формула прямоугольников

$$\int[a; b] f(x) dx = h * \sum_{i=0}^{n-1} f(x_i + h/2) + R_1;$$

Формула трапеций

$$\int[a; b] f(x) dx = (h/2) * \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1})) + R_2;$$

Формула Симпсона

$$\int[a; b] f(x) dx = (h/3) * \sum_{i=0}^{m-1} (f(x_{2i}) + 4 * f(x_{2i+1}) + f(x_{2i+2})) + R_3,$$

где  $R_1, R_2, R_3$  — остаточные члены. При  $n \rightarrow \infty$  приближенные значения интегралов для всех трех формул, в предположении отсутствия погрешностей округления, стремятся к точному значению интеграла [1, 7, 8].

Для практической оценки погрешности квадратурной формулы можно использовать правило Рунге. Для этого проводят вычисления на сетках с шагом  $h$  и  $h/2$ , получают приближенные значения интеграла  $I_h$  и  $I_{h/2}$  и за окончательные значения интеграла принимают величины:

$$I = I_{h/2} + (I_{h/2} - I_h) / 3 \text{ — для формулы прямоугольников;}$$

$$I = I_{h/2} + (I_{h/2} - I_h) / 3 \text{ — для формулы трапеций;}$$

$$I = I_{h/2} + (I_{h/2} - I_h) / 15 \text{ — для формулы Симпсона.}$$

За погрешность приближенного значения интеграла для формул прямоугольников и трапеций тогда принимают величину:  $|I_{h/2} - I_h| / 3$ ,

а для формулы Симпсона:  $|I_{h/2} - I_h| / 15$ .

В лабораторной работе №6 требуется, используя квадратурные формулы прямоугольников, трапеций и Симпсона, вычислить значения заданного интеграла и, применив правило Рунге, найти наименьшее значение  $n$ , то есть наибольшее значение шага  $h$ , при котором каждая из указанных формул дает приближенное значение интеграла с погрешностью  $\varepsilon$ , не превышающей заданную.

Порядок выполнения лабораторной работы №6.

- 1) Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.
- 2) Составить программу-функцию для вычисления подынтегральной функции.
- 3) Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих  $n$  до тех пор, пока погрешность не станет меньше  $\varepsilon$ , и осуществляющих печать результатов: значения интеграла и значения  $n$  для каждой формулы.
- 4) Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности.
- 5) Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

Варианты заданий приведены в таблице 4.1.  $\varepsilon = 0.01; 0.001; 0.0001$ .

Вариант 16:  $f(x) = \int_{\pi/4}^{\pi/2} \ln \sin x \, dx$

## **Выполнение работы**

### **1. Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона**

На первом этапе выполнения лабораторной работы были реализованы три отдельные программы-функции для численного вычисления определенного интеграла: по составной формуле прямоугольников, по составной формуле трапеций и по составной формуле Симпсона. Разделение на отдельные функции сделано для того, чтобы каждый метод можно было вызывать независимо и затем сравнивать полученные результаты между собой.

Для всех трех методов используется общий подход: отрезок интегрирования  $[a, b]$  разбивается на  $n$  равных частей, после чего вычисляется шаг сетки:  $h = (b - a) / n$

В функции для метода прямоугольников реализована составная формула средних прямоугольников. На каждом малом отрезке выбирается его середина, в этой точке вычисляется значение подынтегральной функции, затем все найденные значения суммируются и умножаются на шаг  $h$ . Такой способ позволяет заменить площадь под графиком функции суммой площадей прямоугольников.

В функции для метода трапеций реализована составная формула трапеций. В этом методе на каждом малом отрезке график функции приближенно заменяется прямой линией, соединяющей значения функции на концах отрезка. Значения функции в крайних точках всего промежутка учитываются с половинным весом, а значения во внутренних узлах сетки — с полным весом. После суммирования результат также умножается на шаг  $h$ .

В функции для метода Симпсона реализована составная формула Симпсона. Данный метод использует более точное приближение, так как на парах соседних отрезков график функции заменяется параболой. Внутренние значения функции учитываются с чередующимися коэффициентами 4 и 2, а значения на концах отрезка — с коэффициентом 1. Так как формула Симпсона

применяется на парах отрезков, в программе учитывается требование, что число разбиений  $n$  должно быть четным.

Таким образом, в программе были подготовлены три основные функции численного интегрирования. Они принимают одинаковые исходные данные: границы интегрирования и количество отрезков разбиения. Это позволяет далее использовать их в общей программе для оценки точности по правилу Рунге и определения минимального значения  $n$ , при котором достигается заданная погрешность.

## **2. Составить программу-функцию для вычисления подынтегральной функции**

На втором этапе выполнения лабораторной работы была составлена отдельная программа-функция для вычисления значения подынтегральной функции. По условию варианта требуется вычислить определенный интеграл:

$$\int \text{от } \pi/4 \text{ до } \pi/2 \ln(\sin(x)) \, dx$$

Следовательно, подынтегральная функция имеет вид:  $f(x) = \ln(\sin(x))$

Для вычисления этой функции в программе была реализована отдельная функция  $f(x)$ , которая получает на вход значение аргумента  $x$  и возвращает значение выражения  $\ln(\sin(x))$ .

При вычислении сначала находится значение  $\sin(x)$ , затем от полученного результата берется натуральный логарифм. Так как интегрирование выполняется на отрезке  $[\pi/4, \pi/2]$ , функция определена на всем промежутке корректно: значение  $\sin(x)$  на этом отрезке положительно, поэтому логарифм существует.

Выделение подынтегральной функции в отдельную программу-функцию делает реализацию более удобной и универсальной. Все квадратурные формулы обращаются к одной и той же функции  $f(x)$ , поэтому при необходимости изменить подынтегральное выражение достаточно изменить только одну часть программы, не переписывая сами методы прямоугольников, трапеций и Симпсона.

## **3. Составить головную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул,**

**удваивающих  $n$  до тех пор, пока погрешность не станет меньше  $\varepsilon$ , и осуществляющих печать результатов: значения интеграла и значения  $n$  для каждой формулы**

На третьем этапе была составлена головная программа, которая объединяет ранее реализованные функции численного интегрирования и выполняет автоматический подбор количества разбиений  $n$  для каждой квадратурной формулы. Основная задача этой части программы заключается в том, чтобы определить минимальное значение  $n$ , при котором оценка погрешности становится меньше заданной точности  $\varepsilon$ .

В головной программе задаются пределы интегрирования:

$$a = \pi / 4$$

$$b = \pi / 2$$

Также задаются значения требуемой точности:

$$\varepsilon = 0.01$$

$$\varepsilon = 0.001$$

$$\varepsilon = 0.0001$$

Для каждой из этих точностей программа последовательно выполняет вычисления по формуле прямоугольников, формуле трапеций и формуле Симпсона.

Проверка точности выполняется с помощью правила Рунге. Сначала интеграл вычисляется на сетке с некоторым количеством отрезков  $n$ , затем вычисление повторяется на более мелкой сетке, где количество отрезков увеличено в два раза, то есть используется  $2n$ . После этого сравниваются два полученных приближенных значения интеграла.

Для формул прямоугольников и трапеций оценка погрешности вычисляется как:  $|I(2n) - I(n)| / 3$

Для формулы Симпсона оценка погрешности вычисляется как:

$$|I(2n) - I(n)| / 15$$

Различие в знаменателях связано с порядком точности методов. Формулы прямоугольников и трапеций имеют второй порядок точности, поэтому используется число 3. Формула Симпсона имеет четвертый порядок точности, поэтому используется число 15.

Если найденная оценка погрешности больше заданного значения  $\varepsilon$ , программа снова удваивает количество разбиений  $n$  и повторяет вычисления. Этот процесс продолжается до тех пор, пока не выполнится условие: погрешность  $\leq \varepsilon$

После достижения требуемой точности программа фиксирует найденное значение  $n$ , соответствующий шаг  $h$ , приближенное значение интеграла и оценку погрешности по правилу Рунге.

Для каждого метода головная программа выводит на экран следующие данные: название используемой квадратурной формулы, заданную точность  $\varepsilon$ , найденное количество разбиений  $n$ , значение шага  $h$ , приближенное значение интеграла и оценку погрешности. Это позволяет сравнить методы между собой и определить, какая формула достигает заданной точности при меньшем числе разбиений.

Таким образом, головная программа обеспечивает автоматическое применение правила Рунге, последовательное уточнение сетки путем удвоения  $n$  и вывод итоговых результатов для всех трех методов численного интегрирования.

#### **4. Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности**

На четвертом этапе были проведены вычисления по составленной программе. Для заданного интеграла  $\int_{\pi/4}^{\pi/2} \ln(\sin(x)) dx$  были использованы три квадратурные формулы: формула прямоугольников, формула трапеций и формула Симпсона.

Вычисления проводились для трех значений точности:  $\varepsilon = 0.01$ ,  $\varepsilon = 0.001$ ,  $\varepsilon = 0.0001$



Для каждой формулы программа начинала вычисления с малого количества разбиений  $n$ , затем последовательно удваивала это значение. На каждом шаге интеграл вычислялся дважды: сначала при текущем значении  $n$ , затем при удвоенном количестве разбиений. После этого по правилу Рунге оценивалась погрешность результата.

Если оценка погрешности была больше заданного значения  $\varepsilon$ , количество разбиений снова увеличивалось в два раза. Процесс продолжался до тех пор, пока оценка погрешности не становилась меньше или равной заданной точности.

В результате были получены следующие значения:

| $\varepsilon$ | Метод          | $n$ | $h$            | Значение интеграла | Оценка погрешности |
|---------------|----------------|-----|----------------|--------------------|--------------------|
| 0.01          | Прямоугольники | 4   | 0.196349540849 | -0.084814429511    | 0.001572880633     |
| 0.01          | Трапеции       | 4   | 0.196349540849 | -0.089618374887    | 0.003174195758     |
| 0.01          | Симпсон        | 4   | 0.196349540849 | -0.086444179129    | 0.000025159569     |
| 0.001         | Прямоугольники | 8   | 0.098174770425 | -0.086012579569    | 0.000399383352     |
| 0.001         | Трапеции       | 8   | 0.098174770425 | -0.087216402199    | 0.000800657563     |
| 0.001         | Симпсон        | 4   | 0.196349540849 | -0.086444179129    | 0.000025159569     |
| 0.0001        | Прямоугольники | 32  | 0.024543692606 | -0.086388627549    | 0.000025090894     |
| 0.0001        | Трапеции       | 32  | 0.024543692606 | -0.086463922876    | 0.000050189336     |
| 0.0001        | Симпсон        | 4   | 0.196349540849 | -0.086444179129    | 0.000025159569     |

По результатам вычислений видно, что все три метода позволяют получить приближенное значение интеграла с заданной точностью. При этом формула Симпсона достигает требуемой точности уже при меньшем количестве разбиений. Это объясняется тем, что формула Симпсона имеет более высокий порядок точности по сравнению с формулами прямоугольников и трапеций.

Для наиболее строгой точности  $\varepsilon = 0.0001$  формулы прямоугольников и трапеций потребовали  $n = 32$ , а формула Симпсона уже при  $n = 4$  дала погрешность меньше заданной. Следовательно, в данной задаче формула Симпсона оказалась наиболее эффективной.

## **5. Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул**

В ходе выполнения лабораторной работы было проведено численное вычисление интеграла  $\int$  от  $\pi/4$  до  $\pi/2$   $\ln(\sin(x)) dx$  с использованием трех составных квадратурных формул: формулы прямоугольников, формулы трапеций и формулы Симпсона. Для оценки точности результатов применялось правило Рунге. Расчеты выполнялись для трех значений требуемой точности:  $\text{eps} = 0.01$ ,  $\text{eps} = 0.001$  и  $\text{eps} = 0.0001$ .

По полученным данным видно, что все три метода позволяют найти значение интеграла с заданной точностью, однако эффективность методов различается. Главным критерием сравнения является количество разбиений  $n$ , которое потребовалось для достижения заданной погрешности. Чем меньше значение  $n$ , тем меньше вычислений нужно выполнить программе, следовательно, тем эффективнее метод.

Для точности  $\text{eps} = 0.01$  все три метода достигли нужной точности уже при  $n = 4$ . Однако оценки погрешности у методов заметно отличаются. У формулы прямоугольников погрешность составила 0.001572880633, у формулы трапеций — 0.003174195758, а у формулы Симпсона — всего 0.000025159569. Это показывает, что даже при одинаковом числе разбиений формула Симпсона дает значительно более точный результат.

Для точности  $\text{eps} = 0.001$  различие между методами становится более заметным. Формуле прямоугольников и формуле трапеций потребовалось увеличить число разбиений до  $n = 8$ , то есть выполнить два удвоения  $n$ . При этом формула Симпсона снова достигла требуемой точности уже при  $n = 4$ . Оценка погрешности Симпсона осталась равной 0.000025159569, что намного меньше заданного значения 0.001.

Для наиболее строгой точности  $\text{eps} = 0.0001$  формулы прямоугольников и трапеций потребовали уже  $n = 32$ . Это означает, что программе пришлось выполнить четыре удвоения количества разбиений. Формула Симпсона при этом

снова удовлетворила требуемой точности при  $n = 4$ . Это является главным подтверждением того, что формула Симпсона в данной задаче оказалась наиболее эффективной.

Сравнение значений  $n$  можно представить следующим образом:

| Точность $\epsilon_{rs}$ | Прямоугольники | Трапеции | Симпсон |
|--------------------------|----------------|----------|---------|
| 0.01                     | $n = 4$        | $n = 4$  | $n = 4$ |
| 0.001                    | $n = 8$        | $n = 8$  | $n = 4$ |
| 0.0001                   | $n = 32$       | $n = 32$ | $n = 4$ |

Из таблицы видно, что при ужесточении требования к точности методы прямоугольников и трапеций требуют значительного увеличения числа разбиений. При переходе от  $\epsilon_{rs} = 0.01$  к  $\epsilon_{rs} = 0.0001$  количество разбиений для этих методов выросло с 4 до 32. У формулы Симпсона число разбиений во всех трех случаях осталось равным 4.

Также можно сравнить оценки погрешности при одинаковом значении  $n = 4$  для точности  $\epsilon_{rs} = 0.01$ :

| Метод          | $n$ | Оценка погрешности |
|----------------|-----|--------------------|
| Прямоугольники | 4   | 0.001572880633     |
| Трапеции       | 4   | 0.003174195758     |
| Симпсон        | 4   | 0.000025159569     |

По этим данным видно, что формула Симпсона дает ошибку примерно в десятки раз меньше, чем формула прямоугольников и формула трапеций. Это связано с тем, что формула Симпсона имеет более высокий порядок точности. Формулы прямоугольников и трапеций имеют второй порядок точности, а формула Симпсона — четвертый порядок точности. Поэтому при уменьшении шага сетки результат по формуле Симпсона уточняется быстрее.

Интересно также сравнить сами приближенные значения интеграла. При самой строгой точности  $\text{eps} = 0.0001$  были получены следующие уточненные значения по Рунге:

| Метод          | I с уточнением по Рунге |
|----------------|-------------------------|
| Прямоугольники | -0.086413718443         |
| Трапеции       | -0.086413733540         |
| Симпсон        | -0.086419019560         |

Значения, полученные по формулам прямоугольников и трапеций при  $n = 32$ , практически совпадают между собой. Это говорит о том, что оба метода при достаточно мелком разбиении сходятся к одному и тому же значению интеграла. Формула Симпсона уже при малом числе разбиений дает значение, близкое к этим результатам, причем ее оценка погрешности также удовлетворяет заданному условию.

Формула прямоугольников в данной задаче дает значение интеграла, которое по модулю сначала получается немного меньше, чем уточненное значение. Например, при  $\text{eps} = 0.01$  значение по формуле равно -0.084814429511, а уточненное по Рунге — -0.086387310144. Формула трапеций, наоборот, при грубом разбиении дает значение более отрицательное: -0.089618374887, а после уточнения по Рунге получается -0.086444179129. То есть методы прямоугольников и трапеций приближаются к результату с разных сторон, что хорошо видно по численным данным.

Формула Симпсона показывает наилучшее поведение: уже при  $n = 4$  она дает значение -0.086444179129, а уточнение по Рунге дает -0.086419019560. При этом оценка погрешности равна 0.000025159569, что меньше даже самого строгого значения  $\text{eps} = 0.0001$ . Следовательно, дальнейшее увеличение  $n$  для формулы Симпсона в рамках заданных точностей не требуется.

Таким образом, по результатам вычислений можно сделать вывод, что все рассмотренные квадратурные формулы применимы для вычисления данного интеграла, но их эффективность различна. Формула трапеций и формула

прямоугольников требуют увеличения числа разбиений при повышении точности. Формула Симпсона обеспечивает наибольшую точность при минимальном количестве разбиений. Поэтому для данного интеграла наиболее эффективной оказалась составная формула Симпсона.

## Вывод

В ходе лабораторной работы были изучены составные квадратурные формулы для численного вычисления определенного интеграла на примере интеграла  $\int$  от  $\pi/4$  до  $\pi/2$   $\ln(\sin(x)) dx$ . Были реализованы подпрограмма вычисления подынтегральной функции  $f(x) = \ln(\sin(x))$ , составная формула прямоугольников, составная формула трапеций и составная формула Симпсона. Для практической оценки точности было применено правило Рунге: интеграл вычислялся на сетках с шагами  $h$  и  $h/2$ , после чего по разности полученных значений определялась оценка погрешности. Проведены вычисления для значений  $\text{eps} = 0.01$ ,  $\text{eps} = 0.001$  и  $\text{eps} = 0.0001$ , при этом количество разбиений  $n$  автоматически удваивалось до тех пор, пока оценка погрешности не становилась меньше заданной точности. По результатам вычислений установлено, что все три формулы позволяют получить значение интеграла с требуемой точностью, однако их эффективность различается. Формулы прямоугольников и трапеций при уменьшении  $\text{eps}$  потребовали увеличения числа разбиений до  $n = 32$ , тогда как формула Симпсона во всех рассмотренных случаях достигла требуемой точности уже при  $n = 4$ . Это подтвердило более высокую точность формулы Симпсона и показало, что для данного интеграла она является наиболее эффективной из рассмотренных квадратурных формул.

## ПРИЛОЖЕНИЕ А ИСХОДНЫЙ КОД ПРОГРАММЫ

### Название файла: main.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <functional>
#include <vector>
#include <string>

/*
    Лабораторная работа №9

    Тема:
    Численное интегрирование.
    Составные квадратурные формулы прямоугольников, трапеций и Симпсона.
    Оценка погрешности по правилу Рунге.

    Вариант:
    Интеграл от  $\pi/4$  до  $\pi/2$ :
         $\ln(\sin(x)) \, dx$ 

    Требуется:
    Для  $\epsilon = 0.01, 0.001, 0.0001$  найти значение интеграла
    и минимальное  $n$ , при котором оценка погрешности не превышает  $\epsilon$ .
*/

// Константа  $\pi$ .
//  $\text{acos}(-1.0)$  — стандартный способ получить число  $\pi$  с высокой точностью.
const double PI = std::acos(-1.0);

/*
    Подынтегральная функция.

    По условию варианта:
         $f(x) = \ln(\sin(x))$ 

    На отрезке  $[\pi/4; \pi/2]$  функция определена корректно,
    потому что  $\sin(x) > 0$  на всем этом отрезке.
*/
double f(double x) {
    return std::log(std::sin(x));
}

/*
    Составная формула прямоугольников.

    Используется формула средних прямоугольников:
         $I \approx h \cdot \sum f(a + (i + 0.5) \cdot h), i = 0, 1, \dots, n - 1$ 

    Здесь:
         $a, b$  — границы интегрирования;
         $n$  — количество частей разбиения;
         $h = (b - a) / n$  — шаг сетки.

    Важно:
    Берется середина каждого маленького отрезка.
```

```

        Именно поэтому точка имеет вид:
            x = a + (i + 0.5) * h
*/
double rectangleFormula(double a, double b, int n) {
    // Находим шаг сетки.
    double h = (b - a) / n;

    // Переменная для накопления суммы значений функции.
    double sum = 0.0;

    // Проходим по всем n маленьким отрезкам.
    for (int i = 0; i < n; ++i) {
        // Середина текущего отрезка.
        double x = a + (i + 0.5) * h;

        // Добавляем значение функции в середине отрезка.
        sum += f(x);
    }

    // Умножаем сумму на h и получаем приближенное значение интеграла.
    return h * sum;
}

/*
    Составная формула трапеций.

    Формула:
        
$$I \approx h * ( (f(a) + f(b)) / 2 + \text{сумма } f(a + i*h), i = 1, 2, \dots, n - 1 )$$


    Идея:
    На каждом маленьком отрезке график функции заменяется прямой линией,
    поэтому площадь считается как площадь трапеции.
*/
double trapezoidFormula(double a, double b, int n) {
    // Находим шаг сетки.
    double h = (b - a) / n;

    // В формуле трапеций значения на концах отрезка берутся с
    коэффициентом 1/2.
    double sum = (f(a) + f(b)) / 2.0;

    // Внутренние точки сетки берутся с коэффициентом 1.
    for (int i = 1; i < n; ++i) {
        double x = a + i * h;
        sum += f(x);
    }

    // Умножаем итоговую сумму на h.
    return h * sum;
}

/*
    Составная формула Симпсона.

    Формула:
        
$$I \approx h/3 * (f(x_0) + 4*f(x_1) + 2*f(x_2) + 4*f(x_3) + \dots + 4*f(x_{n-1}) + f(x_n))$$


```



Условия:

Для формулы Симпсона количество отрезков  $n$  обязательно должно быть четным.

В программе  $n$  всегда начинается с 2 и далее только удваивается, поэтому  $n$  всегда остается четным.

```
*/  
double simpsonFormula(double a, double b, int n) {  
    // На всякий случай проверяем четность n.  
    // Если вдруг передали нечетное n, увеличиваем его на 1.  
    // В основной программе такого быть не должно.  
    if (n % 2 != 0) {  
        ++n;  
    }  
  
    // Находим шаг сетки.  
    double h = (b - a) / n;  
  
    // Начинаем сумму с крайних значений f(a) и f(b).  
    double sum = f(a) + f(b);  
  
    // Обрабатываем внутренние точки x1, x2, ..., x(n-1).  
    for (int i = 1; i < n; ++i) {  
        double x = a + i * h;  
  
        // Если индекс нечетный, коэффициент равен 4.  
        if (i % 2 == 1) {  
            sum += 4.0 * f(x);  
        }  
        // Если индекс четный, коэффициент равен 2.  
        else {  
            sum += 2.0 * f(x);  
        }  
    }  
  
    // Умножаем сумму на h/3.  
    return (h / 3.0) * sum;  
}
```

/\*

Структура для хранения результата работы одного метода.

Она нужна, чтобы удобно вернуть из функции сразу несколько значений:  
название метода;

eps;

найденное  $n$ ;

шаг  $h$ ;

приближенное значение интеграла;

уточненное значение по Рунге;

оценку погрешности;

количество удвоений  $n$ .

\*/

```
struct Result {  
    std::string methodName;    // Название метода  
    double eps;                // Заданная точность  
    int n;                     // Найденное количество отрезков  
    double h;                  // Шаг сетки
```

```

double integral;           // Приближенное значение интеграла
double rungeIntegral;      // Уточненное значение по правилу Рунге
double error;              // Оценка погрешности по Рунге
int iterations;            // Сколько раз удваивали n
};

/*
Универсальная функция для применения правила Рунге.

Параметры:
    methodName — название формулы;
    formula — функция, которая считает интеграл выбранным методом;
    order — порядок точности метода;
    a, b — границы интегрирования;
    eps — требуемая точность.

Порядки точности:
    прямоугольники по серединам — 2;
    трапеции — 2;
    Симпсон — 4.

Правило Рунге:

Считаем интеграл на сетке с n отрезками:
    I_old

Потом считаем на сетке с 2n отрезками:
    I_new

Оценка погрешности:
    error = |I_new - I_old| / (2^p - 1)

где p — порядок точности метода.

Уточненное значение:
    I_runge = I_new + (I_new - I_old) / (2^p - 1)

Цикл продолжается до тех пор, пока:
    error <= eps
*/
Result rungeRule(
    const std::string& methodName,
    const std::function<double(double, double, int)>& formula,
    int order,
    double a,
    double b,
    double eps
) {
    // Начинаем с n = 2.
    // Это удобно, потому что формула Симпсона требует четное n.
    int n = 2;

    // Считаем первое приближение на грубой сетке.
    double oldIntegral = formula(a, b, n);

    // Счетчик удвоений n.
    int iterations = 0;

```

```

while (true) {
    // Удваиваем количество отрезков.
    // При этом шаг h уменьшается в 2 раза.
    n *= 2;
    ++iterations;

    // Считаем новое приближение на более мелкой сетке.
    double newIntegral = formula(a, b, n);

    // Знаменатель в правиле Рунге:
    // для методов 2-го порядка:  $2^2 - 1 = 3$ ;
    // для метода Симпсона:  $2^4 - 1 = 15$ .
    double denominator = std::pow(2.0, order) - 1.0;

    // Оценка погрешности по правилу Рунге.
    double error = std::abs(newIntegral - oldIntegral) / denominator;

    // Уточненное значение интеграла по правилу Рунге.
    double rungeIntegral = newIntegral + (newIntegral - oldIntegral)
/ denominator;

    // Если требуемая точность достигнута, возвращаем результат.
    if (error <= eps) {
        Result result;

        result.methodName = methodName;
        result.eps = eps;
        result.n = n;
        result.h = (b - a) / n;
        result.integral = newIntegral;
        result.rungeIntegral = rungeIntegral;
        result.error = error;
        result.iterations = iterations;

        return result;
    }

    // Если точность еще не достигнута,
    // новое значение становится старым,
    // после чего цикл продолжится с еще меньшим шагом.
    oldIntegral = newIntegral;
}
}

/*
Функция печати результата.

Она выводит:
метод;
eps;
найденное n;
шаг h;
значение интеграла;
уточненное значение по Рунге;
оценку погрешности;
количество удвоений n.
*/
void printResult(const Result& result) {

```

```

std::cout << "Метод: " << result.methodName << '\n';
std::cout << "eps = " << result.eps << '\n';
std::cout << "n = " << result.n << '\n';
std::cout << "h = " << result.h << '\n';
std::cout << "I по формуле = " << result.integral << '\n';
std::cout << "I с уточнением по Рунге = " << result.rungeIntegral <<
'\n';
std::cout << "Оценка погрешности по Рунге = " << result.error <<
'\n';
std::cout << "Количество удвоений n = " << result.iterations << '\n';
std::cout << "-----\n";
-----\n";
}

int main() {
    /*
        Задаем границы интегрирования по варианту:

        a = pi / 4
        b = pi / 2
    */
    double a = PI / 4.0;
    double b = PI / 2.0;

    /*
        Задаем точности, которые указаны в лабораторной работе:

        eps = 0.01
        eps = 0.001
        eps = 0.0001
    */
    std::vector<double> epsValues = {0.01, 0.001, 0.0001};

    /*
        Настраиваем формат вывода.

        fixed – выводить числа в обычном десятичном виде;
        setprecision(12) – выводить 12 знаков после запятой.
    */
    std::cout << std::fixed << std::setprecision(12);

    std::cout << "Лабораторная работа №9\n";
    std::cout << "Численное интегрирование\n";
    std::cout << "Интеграл:  $\ln(\sin(x))$  dx на отрезке  $[\pi/4; \pi/2]$ \n";
    std::cout << "-----\n\n";

    /*
        Для каждой точности eps запускаем три метода:
        1) прямоугольники;
        2) трапеции;
        3) Симпсон.

        Для прямоугольников и трапеций порядок точности p = 2.
        Для Симпсона порядок точности p = 4.
    */
    for (double eps : epsValues) {
        std::cout << "Требуемая точность eps = " << eps << "\n";
    }
}

```

```

        std::cout <<
"=====\\n";

        Result rectangleResult = rungeRule(
            "Составная формула прямоугольников",
            rectangleFormula,
            2,
            a,
            b,
            eps
        );

        Result trapezoidResult = rungeRule(
            "Составная формула трапеций",
            trapezoidFormula,
            2,
            a,
            b,
            eps
        );

        Result simpsonResult = rungeRule(
            "Составная формула Симпсона",
            simpsonFormula,
            4,
            a,
            b,
            eps
        );

        printResult(rectangleResult);
        printResult(trapezoidResult);
        printResult(simpsonResult);

        std::cout << "\\n";
    }

    return 0;
}

```